

SIMATS ENGINEERING

ASSESSMENT TOOL 1 : Experiments

CO3_AT1

COURSE NAME: COMPUTER VISION COURSE

COURSE CODE: ITA0518

STUDENT NAME : T. HARI SANKAR

REG . NO:192521160

16: Threshold Effect in Edge Detection

Aim

To design and conduct an experiment that studies the influence of threshold values on the quality of edge detection, using the Canny edge detector, and to analyse how threshold selection affects edge continuity, accuracy and noise sensitivity.

Importance of Thresholding in Edge Detection

- Pixels with gradient magnitude above T_{high} are immediately accepted as 'strong' edge pixels.
- Pixels with gradient magnitude below T_{low} are rejected outright as non-edges (usually noise or smooth regions).
- Pixels between T_{low} and T_{high} are accepted only if they are connected to a strong edge pixel (this preserves continuity of weak-but-genuine edges while still rejecting isolated noise).

Experimental Design

1. Generate/acquire a test image containing objects of different contrast, including one low-contrast (faint) object, to test sensitivity.
2. Convert the image to grayscale and apply a small amount of Gaussian noise to simulate a real camera sensor.
3. Smooth the noisy image slightly (3x3 Gaussian blur) as a standard pre-processing step for Canny.
4. Apply the Canny detector with four increasing threshold pairs: (10,40), (30,80), (60,150), (150,220).
5. Record the resulting edge maps and count the number of edge pixels produced by each threshold pair.
6. Compare the maps visually and quantitatively to study continuity, accuracy and noise sensitivity.

Program

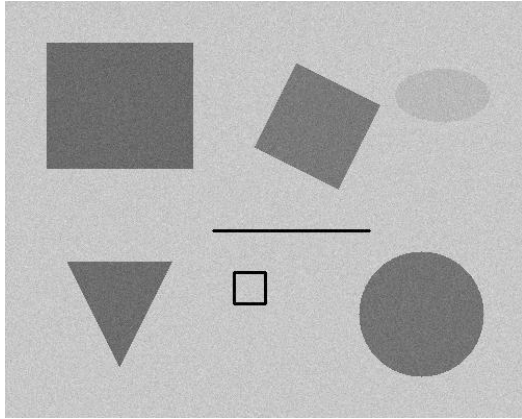
```
import cv2
import numpy as np

img = cv2.imread("base_image.png")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

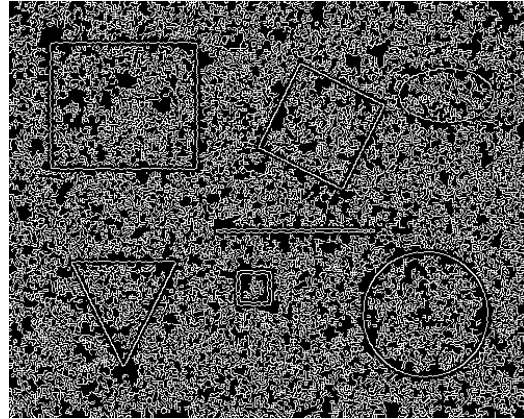
# simulate sensor noise
noise = np.random.normal(0, 8, gray.shape).astype(np.int16)
gray_noisy = np.clip(gray.astype(np.int16) + noise, 0, 255).astype(np.uint8)
gray_blur = cv2.GaussianBlur(gray_noisy, (3, 3), 0)

thresholds = [(10, 40), (30, 80), (60, 150), (150, 220)]
for (lo, hi) in thresholds:
    edges = cv2.Canny(gray_blur, lo, hi)
    print((lo, hi), "-> edge pixels:", np.count_nonzero(edges))
    cv2.imwrite(f'canny_{lo}_{hi}.png', edges)
```

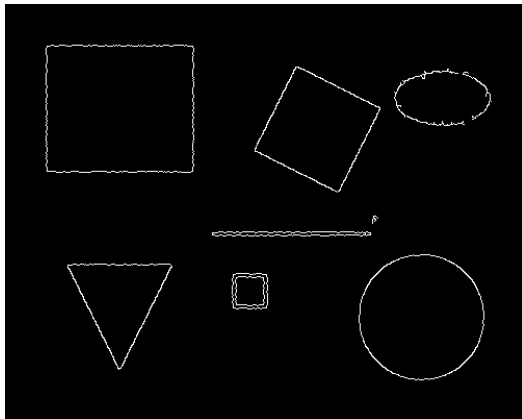
Input and Output Images



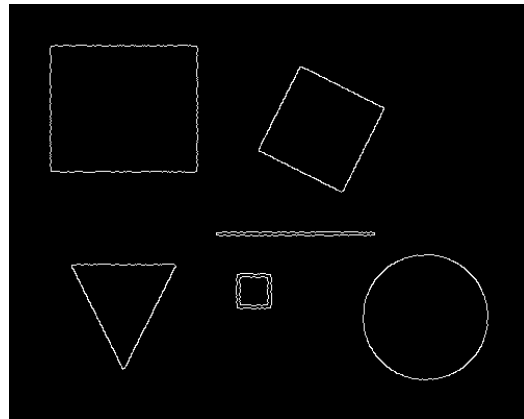
Input: grayscale image with simulated sensor noise



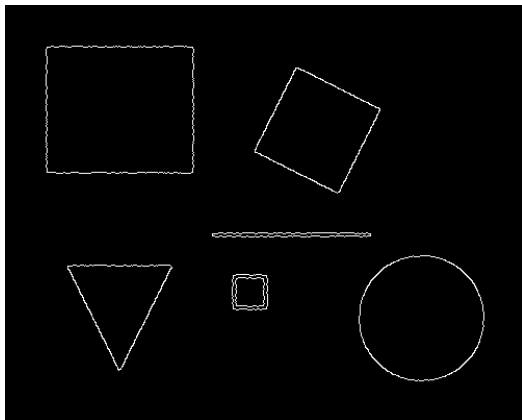
Canny ($T_{low}=10$, $T_{high}=40$) - very low threshold



Canny ($T_{low}=30$, $T_{high}=80$) - moderate threshold



Canny ($T_{low}=60$, $T_{high}=150$) - clean threshold



Canny ($T_{low}=150$, $T_{high}=220$) - very high threshold

Result and Analysis

The lowest threshold pair (10, 40) accepted almost every gradient fluctuation as an edge, so the entire background is covered with spurious noisy edges even though the real object boundaries are also present. This demonstrates poor noise sensitivity.

17: Noise Removal Comparison

Aim

To perform a comparative study of commonly used noise-removal (filtering) techniques - Mean filter, Gaussian filter, Median filter and Bilateral filter - applied to images corrupted with Gaussian noise and Salt-and-Pepper noise, and to analyse their effectiveness in removing noise while preserving image detail.

Objectives and Principles of the Filters

- Mean (Box) Filter: replaces every pixel with the arithmetic average of its neighbourhood. Simple and fast, but blurs edges and cannot distinguish noise from genuine detail.
- Gaussian Filter: a weighted average using a Gaussian kernel, giving more weight to nearby pixels. Produces smoother results than the mean filter and is well suited to additive Gaussian (sensor) noise, but still blurs edges somewhat.
- Median Filter: replaces each pixel with the median value of its neighbourhood. It is a non-linear, order-statistic filter that is extremely effective against impulse (salt-and-pepper) noise because a single extreme outlier does not affect the median, while edges are largely preserved.
- Bilateral Filter: a weighted average like the Gaussian filter, but the weights depend on both spatial distance and intensity difference. This lets it smooth flat regions while preserving strong edges, making it very effective for Gaussian noise while retaining sharpness.

Program

```
import cv2
import numpy as np

img = cv2.imread("base_image.png")

def add_gaussian_noise(image, sigma=25):
    noise = np.random.normal(0, sigma, image.shape).astype(np.float32)
    return np.clip(image.astype(np.float32) + noise, 0, 255).astype(np.uint8)

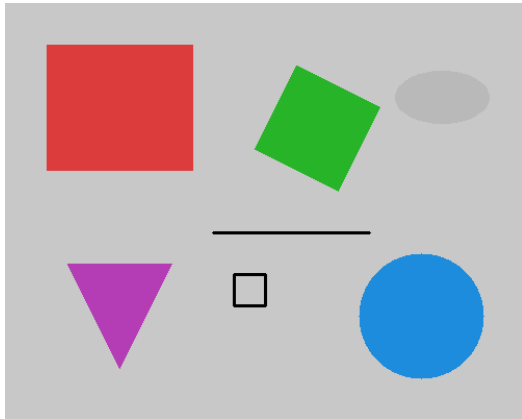
def add_salt_pepper(image, amount=0.04):
    noisy = image.copy()
    h, w = image.shape[:2]
    n = int(amount * h * w * 0.5)
    ys, xs = np.random.randint(0, h, n), np.random.randint(0, w, n)
    noisy[ys, xs] = 255 # salt
    ys, xs = np.random.randint(0, h, n), np.random.randint(0, w, n)
    noisy[ys, xs] = 0 # pepper
    return noisy

def psnr(a, b):
    mse = np.mean((a.astype(np.float64) - b.astype(np.float64)) ** 2)
    return float('inf') if mse == 0 else 20*np.log10(255.0/np.sqrt(mse))

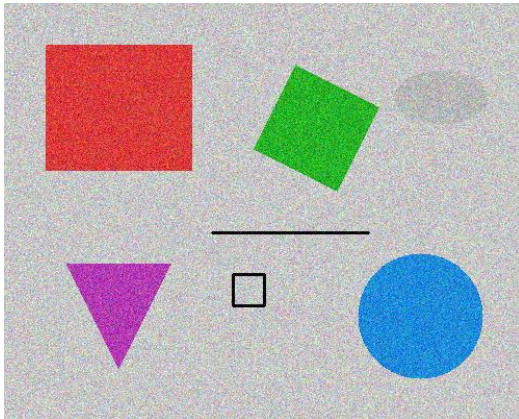
gauss_noisy = add_gaussian_noise(img, 25)
sp_noisy = add_salt_pepper(img, 0.04)

for name, noisy in [("gaussian", gauss_noisy), ("salt_pepper", sp_noisy)]:
    mean_f = cv2.blur(noisy, (5, 5))
    gauss_f = cv2.GaussianBlur(noisy, (5, 5), 1.5)
    median_f = cv2.medianBlur(noisy, 5)
    bilat_f = cv2.bilateralFilter(noisy, 9, 75, 75)
    print(name, "noisy:", round(psnr(img, noisy), 2),
          "mean:", round(psnr(img, mean_f), 2),
          "gaussian:", round(psnr(img, gauss_f), 2),
          "median:", round(psnr(img, median_f), 2),
          "bilateral:", round(psnr(img, bilat_f), 2))
```

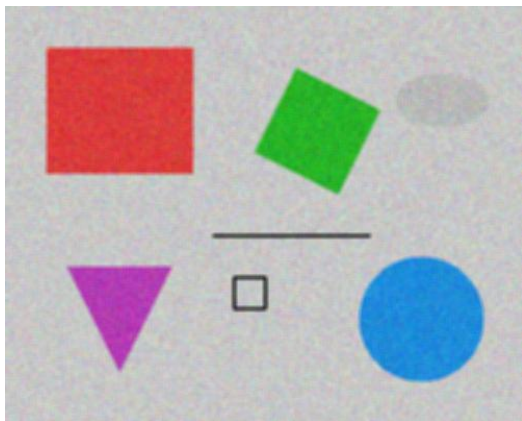
Input and Output Images: Gaussian Noise



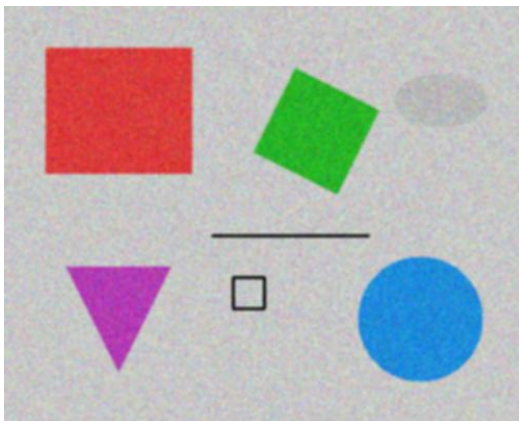
Original clean image



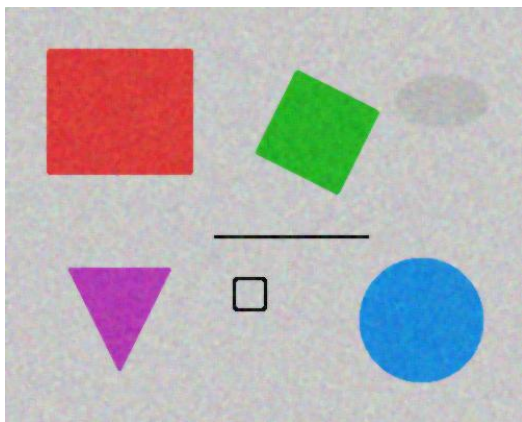
Gaussian noise added ($\sigma = 25$)



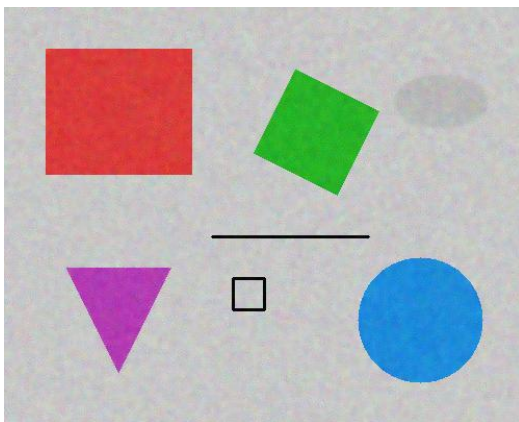
Mean filter (5x5) result



Gaussian filter (5x5, $\sigma=1.5$) result

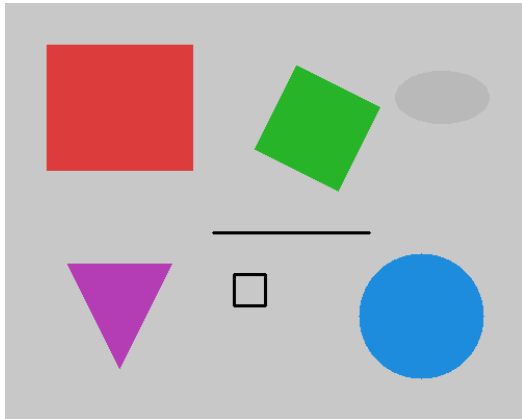


Median filter (5x5) result

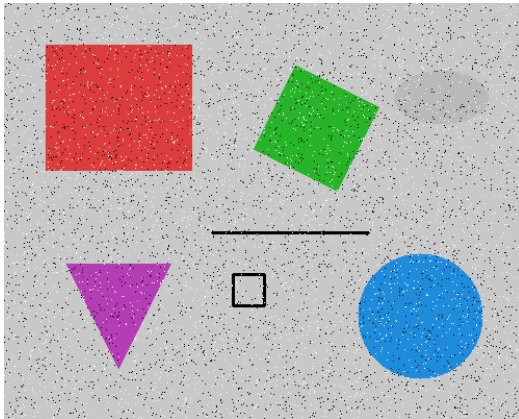


Bilateral filter ($d=9$) result

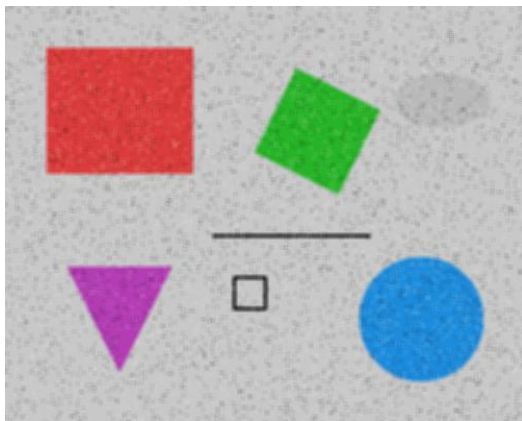
Input and Output Images: Salt-and-Pepper Noise



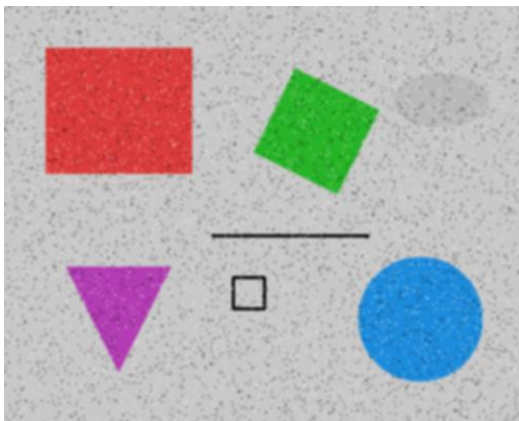
Original clean image



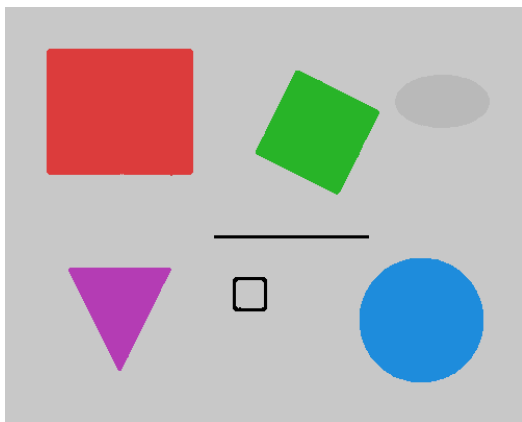
Salt-and-pepper noise added (4%)



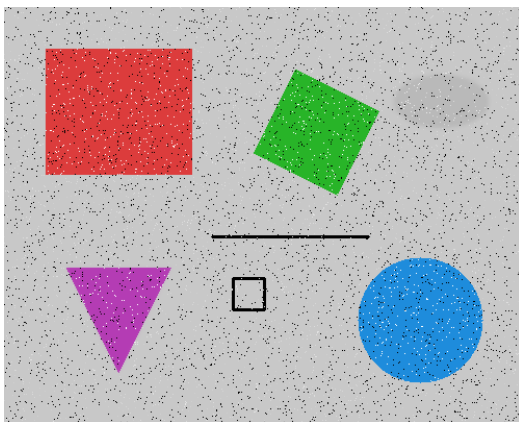
Mean filter (5x5) result



Gaussian filter (5x5, sigma=1.5) result



Median filter (5x5) result



Bilateral filter (d=9) result

Result and Analysis

For Gaussian noise, all four filters improved the PSNR substantially over the noisy image, with the Bilateral filter performing best (31.71 dB) because it averages intensity-similar neighbours while still respecting edges, closely followed by the Median filter. The Mean and Gaussian filters improved noise but visibly softened the object boundaries.

Experiment 18: Feature Detection in Noisy Images

Aim

To conduct an experiment analysing the performance of feature detection (ORB - Oriented FAST and Rotated BRIEF) on images corrupted with increasing levels of noise, and to study the effect of noise on the accuracy and reliability of detected features.

Experimental Design

1. Take a clean test image and create three noisy versions using additive Gaussian noise with $\sigma = 15, 30$ and 50 .
2. Run the ORB detector (max 500 features) on the clean image and on each noisy version; record the number of keypoints detected.
3. Match ORB descriptors between the clean image and the $\sigma=30$ noisy image using a Brute-Force Hamming matcher to measure how many keypoints are still reliably identifiable after noise is added.
4. Visualise and compare the keypoints and the matches.

Program

```
import cv2
import numpy as np

img = cv2.imread("base_image.png")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

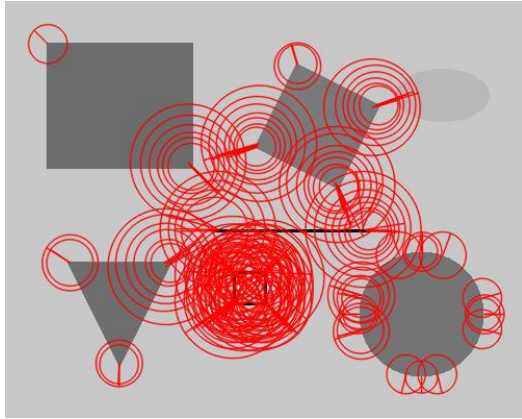
def add_gaussian_noise(image, sigma):
    noise = np.random.normal(0, sigma, image.shape).astype(np.float32)
    return np.clip(image.astype(np.float32) + noise, 0, 255).astype(np.uint8)

orb = cv2.ORB_create(nfeatures=500)

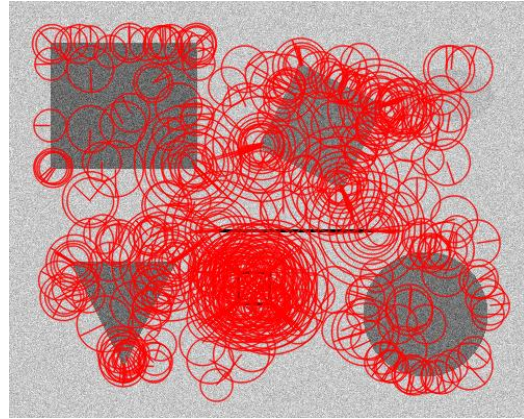
for sigma in [0, 15, 30, 50]:
    noisy = gray if sigma == 0 else add_gaussian_noise(gray, sigma)
    kp, des = orb.detectAndCompute(noisy, None)
    print("sigma =", sigma, "-> keypoints detected:", len(kp))

# repeatability: match clean vs sigma=30
kp0, des0 = orb.detectAndCompute(gray, None)
noisy30 = add_gaussian_noise(gray, 30)
kp1, des1 = orb.detectAndCompute(noisy30, None)
bf = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True)
matches = bf.match(des0, des1)
print("Good matches (clean vs sigma=30):", len(matches))
```

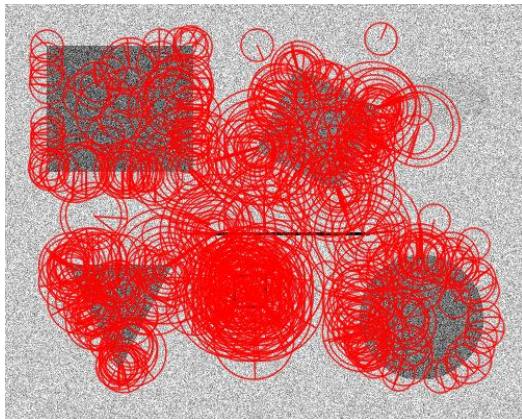

Input and Output Images



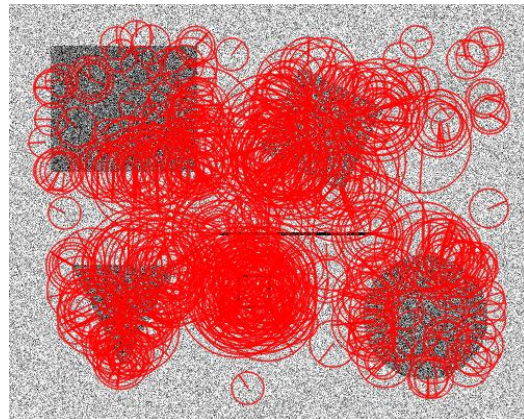
ORB keypoints - clean image (121 keypoints)



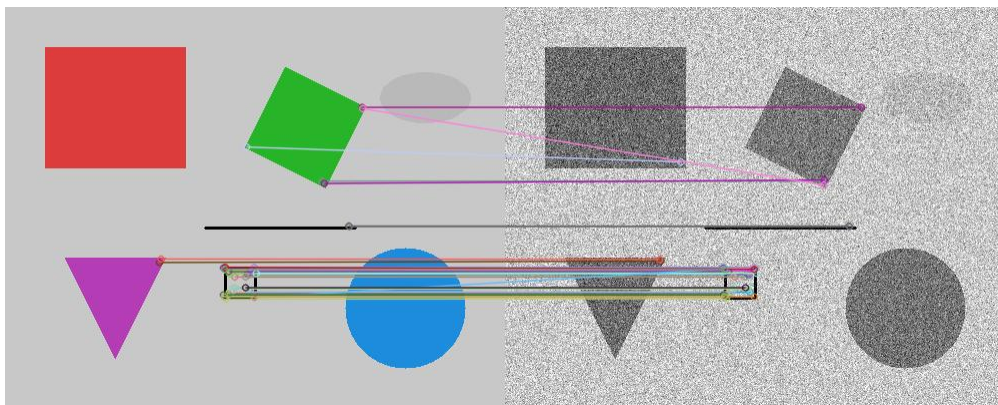
ORB keypoints - sigma=15 noise (363 keypoints)



ORB keypoints - sigma=30 noise (460 keypoints)



ORB keypoints - sigma=50 noise (489 keypoints)



ORB descriptor matches between the clean image and the sigma=30 noisy image (only 51 of the original 121 keypoints could be reliably matched)

Result and Analysis

The number of ORB keypoints increases sharply as noise increases (from 121 at $\sigma=0$ to 489 at $\sigma=50$), rather than decreasing. This is because random noise creates a very large number of small, spurious high-contrast patches that FAST/ORB mistakes for corners, as visible in the keypoint images where the shapes become densely covered with overlapping keypoint circles. However, this larger keypoint count does not indicate better detection - it indicates lower reliability.

19: Edge Detection under Illumination Changes

Aim

To design an experiment studying the effect of illumination variation (underexposure, overexposure and uneven/non-uniform lighting) on edge detection performance, using a fixed-threshold Canny detector.

Relationship between Lighting and Edge Extraction

- Underexposure (too dark): pixel values are compressed near 0; gradients are still present but the image may pick up more visible sensor/quantisation noise relative to the small usable range.
- Overexposure (too bright): pixel values clip / saturate at 255. Any object whose brightened value exceeds 255 loses its distinguishing intensity difference from its equally-clipped neighbours, destroying the gradient and therefore the edge, even though the object is clearly visible to a human eye.
- Uneven/non-uniform illumination (e.g. a lighting gradient across the scene): superimposes a slow, large-scale gradient on top of the true object edges; if strong enough or combined with a low threshold, this can itself be picked up as a false 'edge' or can shift the local contrast of real edges unpredictably.

Experimental Design

1. Take a single grayscale test image and generate four illumination variants using the SAME underlying scene: dark (brightness -150), normal (unchanged), bright (brightness +130, causing saturation), and uneven (a left-to-right linear brightness gradient of range 220).
2. Apply the Canny edge detector with IDENTICAL, FIXED thresholds (60, 150) to all four variants, so that any difference in the result is attributable only to illumination, not to threshold re-tuning.
3. Compare the resulting edge maps visually and by counting edge pixels.

Program

```
import cv2
import numpy as np

gray = cv2.cvtColor(cv2.imread("base_image.png"), cv2.COLOR_BGR2GRAY)

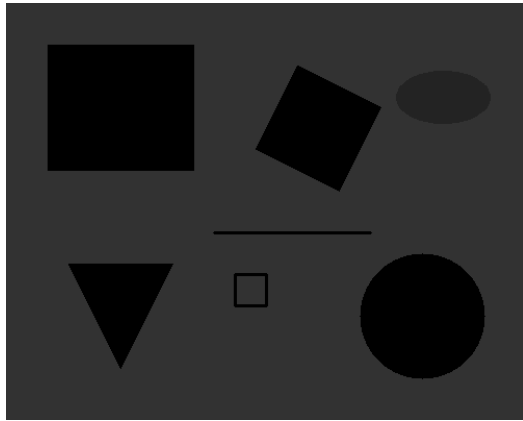
def adjust_brightness(image, beta):
    return np.clip(image.astype(np.int16) + beta, 0, 255).astype(np.uint8)

def add_illumination_gradient(image, strength):
    h, w = image.shape[:2]
    ramp = np.tile(np.linspace(-strength/2, strength/2, w), (h, 1)).astype(np.int16)
    return np.clip(image.astype(np.int16) + ramp, 0, 255).astype(np.uint8)

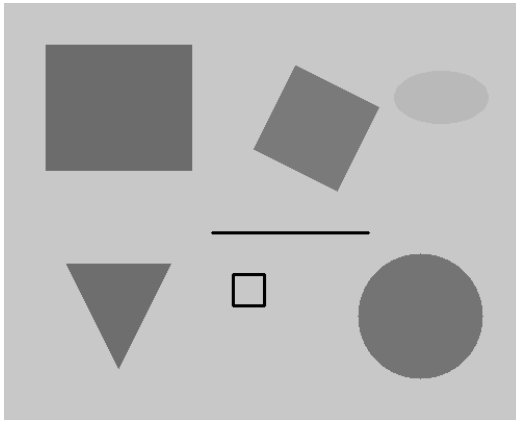
variants = {
    "dark": adjust_brightness(gray, -150),
    "normal": gray,
    "bright": adjust_brightness(gray, 130),
    "uneven": add_illumination_gradient(gray, 220),
}

for name, im in variants.items():
    edges = cv2.Canny(im, 60, 150) # SAME fixed thresholds for every variant
    print(name, "-> edge pixels:", np.count_nonzero(edges))
    cv2.imwrite(f"canny_{name}.png", edges)
```

Input Images (Different Illumination Conditions)



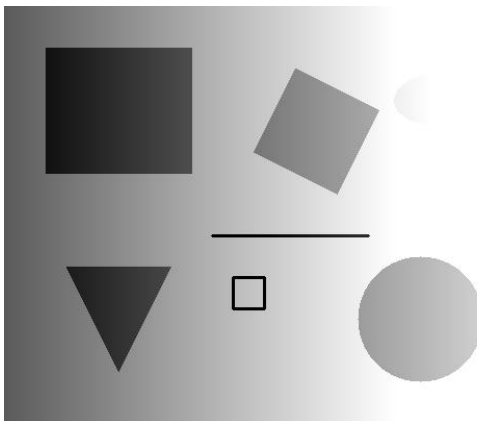
Input: underexposed (beta = -150)



Input: normal illumination

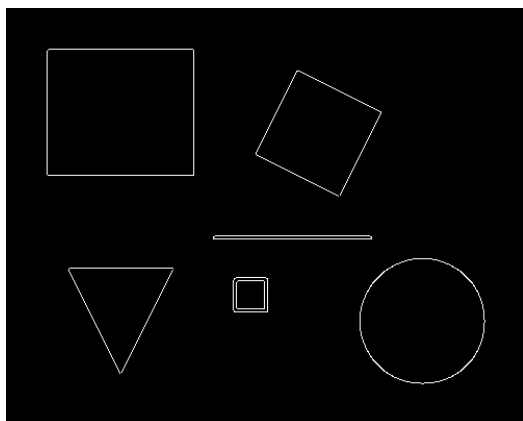


Input: overexposed (beta = +130)

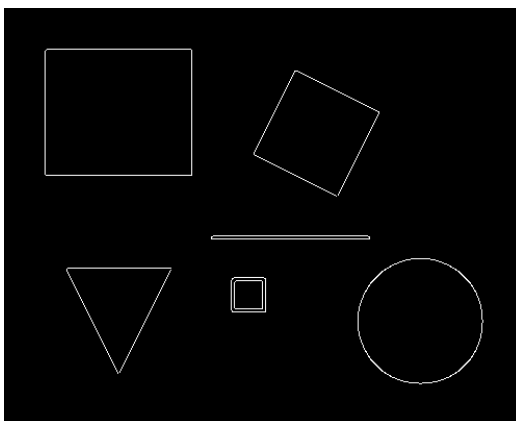


Input: uneven left-right lighting gradient

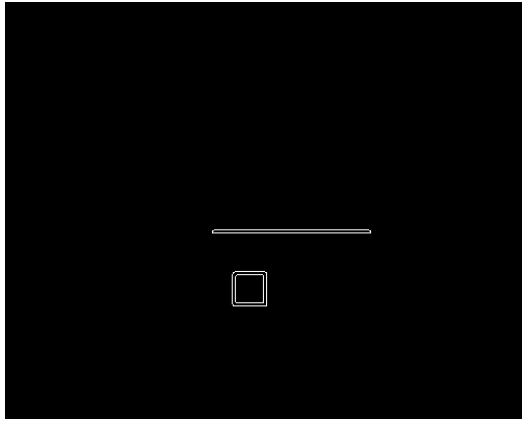
Output Edge Maps (Same Fixed Canny Thresholds)



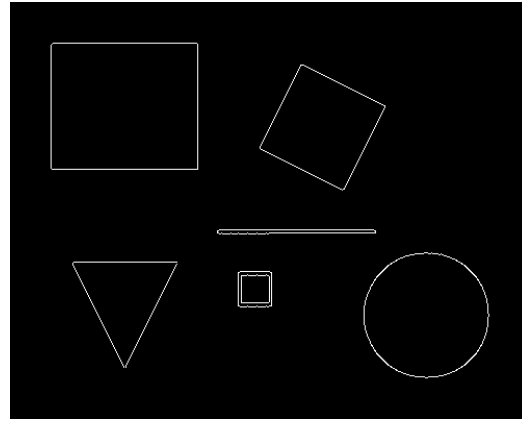
Canny output - dark image (2081 edge px)



Canny output - normal image (2081 edge px)



Canny output - bright/saturated image (540 edge px)



Canny output - uneven illumination (2083 edge px)

Observation Table

Illumination Condition	Edge Pixels	Visual Quality	Reason
Dark (underexposed)	2081	Identical to normal	Gradients preserved; no clipping at 0 for these shapes
Normal	2081	Reference / best	-
Bright (overexposed)	540	Severely degraded - most shapes lost	Saturation at 255 removes intensity contrast between shape and background
Uneven lighting	2083	Same as normal	Linear gradient is too gentle locally to affect the Canny gradient operator

Result and Analysis

The dark and uneven-illumination variants produced almost exactly the same edge map as the normal image (2081-2083 edge pixels), confirming that Canny edge detection is largely invariant to uniform brightness shifts and to slowly-varying illumination gradients, since these operations preserve the local intensity gradient across each true object boundary

20: Corner Detection Parameter Study (Harris)

Aim

To perform a parameter-analysis study of the Harris corner detection algorithm, and to analyse how the block size (neighbourhood size), Sobel aperture size, and the sensitivity parameter k affect corner detection accuracy and sensitivity.

Significance of the Harris Parameters

- `blockSize` - the size of the neighbourhood considered for computing the structure tensor at each pixel. A small `blockSize` makes the detector sensitive to fine, localised corners but more susceptible to noise; a large `blockSize` aggregates gradient information over a wider area, detecting corners more robustly but merging nearby distinct corners and thickening/rounding the detected corner regions.
- `ksize` (Sobel aperture size) - the size of the Sobel kernel used to compute the image gradients (I_x , I_y) that feed into the structure tensor. A larger `ksize` smooths the gradient computation itself, making the detector less sensitive to noise but also less precise/localised.
- k (Harris sensitivity/free parameter, typically 0.04-0.06) - controls the trade-off between detecting corners and detecting edges. A small k makes the detector MORE sensitive, flagging more points (including some edge-like points) as corners; a large k makes the response function more conservative, so only the sharpest, most corner-like points are retained, drastically reducing the number of detected corners.

Experimental Design

1. Use a test image containing well-defined polygonal shapes with known corner locations (rectangle, rotated square, triangle, small square) as ground truth.
2. Run `cv2.cornerHarris` with four parameter configurations, changing one parameter at a time from a baseline of (`blockSize=2`, `ksize=3`, $k=0.04$): (a) baseline, (b) k increased to 0.20, (c) `blockSize` increased to 9, (d) `ksize` increased to 7.
3. Threshold the Harris response at 1% of its maximum value ($R > 0.01 * R_{\max}$) to mark detected corner pixels in red.
4. Count the number of pixels marked as corners in each case and compare visually.

Program

```
import cv2
import numpy as np

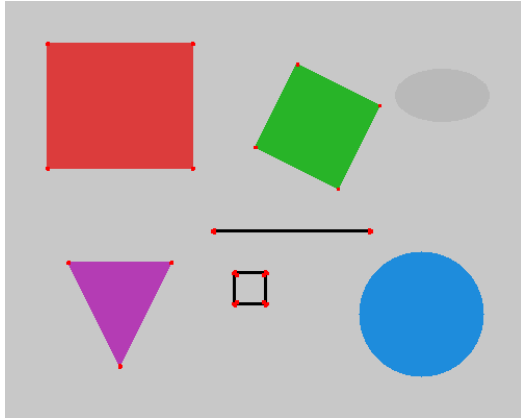
img = cv2.imread("base_image.png")
gray = np.float32(cv2.cvtColor(img, cv2.COLOR_BGR2GRAY))

def run_harris(blockSize, ksize, k, thresh_ratio=0.01):
    dst = cv2.cornerHarris(gray, blockSize, ksize, k)
    dst = cv2.dilate(dst, None)
    mask = dst > thresh_ratio * dst.max()
    vis = img.copy()
    vis[mask] = [0, 0, 255]
    return vis, int(np.count_nonzero(mask))

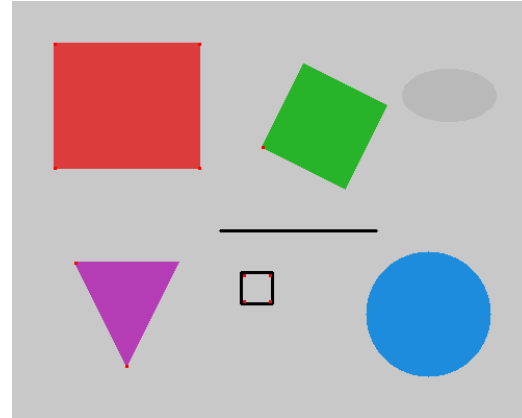
configs = [
    dict(blockSize=2, ksize=3, k=0.04), # baseline
    dict(blockSize=2, ksize=3, k=0.20), # higher k -> more conservative
    dict(blockSize=9, ksize=3, k=0.04), # larger blockSize -> more sensitive/robust
    dict(blockSize=2, ksize=7, k=0.04), # larger Sobel aperture -> smoother gradients
]
```

```
for cfg in configs:
    vis, n = run_harris(**cfg)
    print(cfg, "-> corner pixels:", n)
```

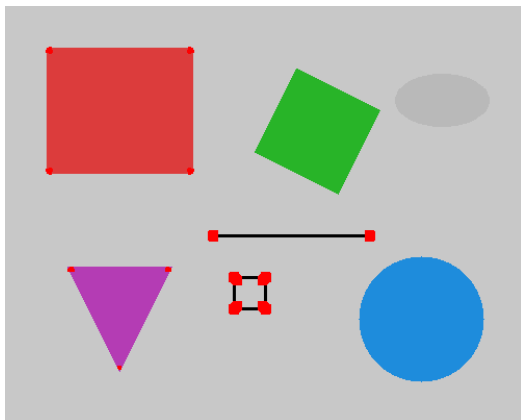
Input and Output Images



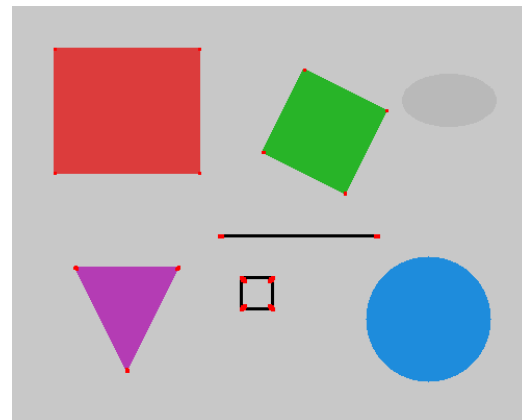
Baseline: blockSize=2, ksize=3, k=0.04 (338 px)



Higher k: blockSize=2, ksize=3, k=0.20 (99 px)



Larger blockSize=9, ksize=3, k=0.04 (1036 px)



Larger ksize=7, blockSize=2, k=0.04 (343 px)

Observation Table

Configuration	blockSize	ksize	k	Corner Pixels	Effect
Baseline	2	3	0.04	338	All true polygon vertices detected, sharply localised
High k	2	3	0.20	99	Fewer corners; only the sharpest vertices survive
Large blockSize	9	3	0.04	1036	Corner regions become thick/rounded; more pixels flagged around each vertex
Large ksize	2	7	0.04	343	Similar count to baseline but slightly smoother/thicker corner blobs

Result and Analysis

The baseline configuration ($\text{blockSize}=2$, $\text{ksize}=3$, $k=0.04$) correctly localises all vertices of the rectangle, rotated square, triangle and small square as sharp, thin corner clusters (338 pixels). Increasing k to 0.20 made the detector far more conservative, reducing detected corner pixels to 99 - only the most unambiguous, high-curvature vertices survive, illustrating that k directly suppresses edge-like or weak-corner responses. Increasing blockSize to 9 nearly tripled the detected corner-pixel count (1036) and visibly thickened the corner markings in the output image, because a larger neighbourhood window causes the structure tensor to respond strongly over a wider area around each true corner (and can also start responding near edges/curved boundaries such as the circle), trading precision for robustness. Increasing the Sobel aperture ($\text{ksize}=7$) gave a corner-pixel count (343) close to the baseline but with visibly smoother/slightly larger corner blobs, since the wider Sobel kernel smooths the gradient estimate, marginally reducing sensitivity to pixel-level noise at the cost of exact localisation.

Conclusion

The three Harris parameters control distinct trade-offs: k trades sensitivity for selectivity (higher k = fewer, more confident corners), blockSize trades localisation precision for robustness (larger blockSize = thicker but more robust responses, and possible false positives on curved edges), and ksize trades gradient noise-robustness for localisation sharpness. For precise corner localisation on relatively clean images, small $\text{blockSize}/\text{ksize}$ with a standard k (~ 0.04 - 0.06) is preferred; for noisier images, a larger blockSize can help but requires careful thresholding to avoid excessive false detections.